```
        gamma * max(q_table[next_state]) - q_table[state][move]
    )

    state = next_state

# Display trained Q-Table for the initial state print("Trained Q-Table for the initial state:")
print(q_table[board_to_string([" "] * 9)])
```

Understanding the Example

1. **State Representation**:
    - The board is represented as a list of nine elements. Empty spaces are denoted by " ", and moves by "X" or "O".
    - The state is converted into a string to use as a key in the Q-Table.
2. **Action Selection**:
    - The AI selects actions (moves) randomly during training to explore different strategies.
    - Q-values guide the agent to prefer moves leading to higher rewards in future games.
3. **Rewards**:
    - Winning provides a positive reward, while losing incurs a penalty.
    - Draws are given a neutral reward to encourage learning toward winning.
4. **Learning Dynamics**:
    - The Q-values are updated iteratively using the Bellman equation, balancing immediate and future rewards.

Applications of Game AI

1. **Learning Optimal Strategies**:
    - Reinforcement Learning enables agents to identify optimal strategies by playing multiple iterations of the game.